
OUTILS JAVA

Master I I.F.R.I

KOUKPAKI Jean-Pierre
Ingénieur informatique / Expert logiciel

SOMMAIRE

1. Introduction au langage Java
2. Les bonnes pratiques de développement en Java
3. Les design patterns
4. Gestion des dépendances projet avec Maven
5. Le framework Spring (Spring web, SpringData, SpringSecurity, SpringBoot...)
6. Développement d'interfaces graphiques avec les JSP
7. Evaluations pratiques

Introduction

Le langage JAVA

Langage de programmation orienté objet

Proche du langage C

Mis au point en 1990 par la société SUN Microsystems

Première présentation en 1995

Dernière version : Java 23 disponible depuis le 17 Septembre 2024

Java 23 fait suite à la LTS Java 21 et précède la prochaine LTS Java 25 prévue pour Septembre 2025

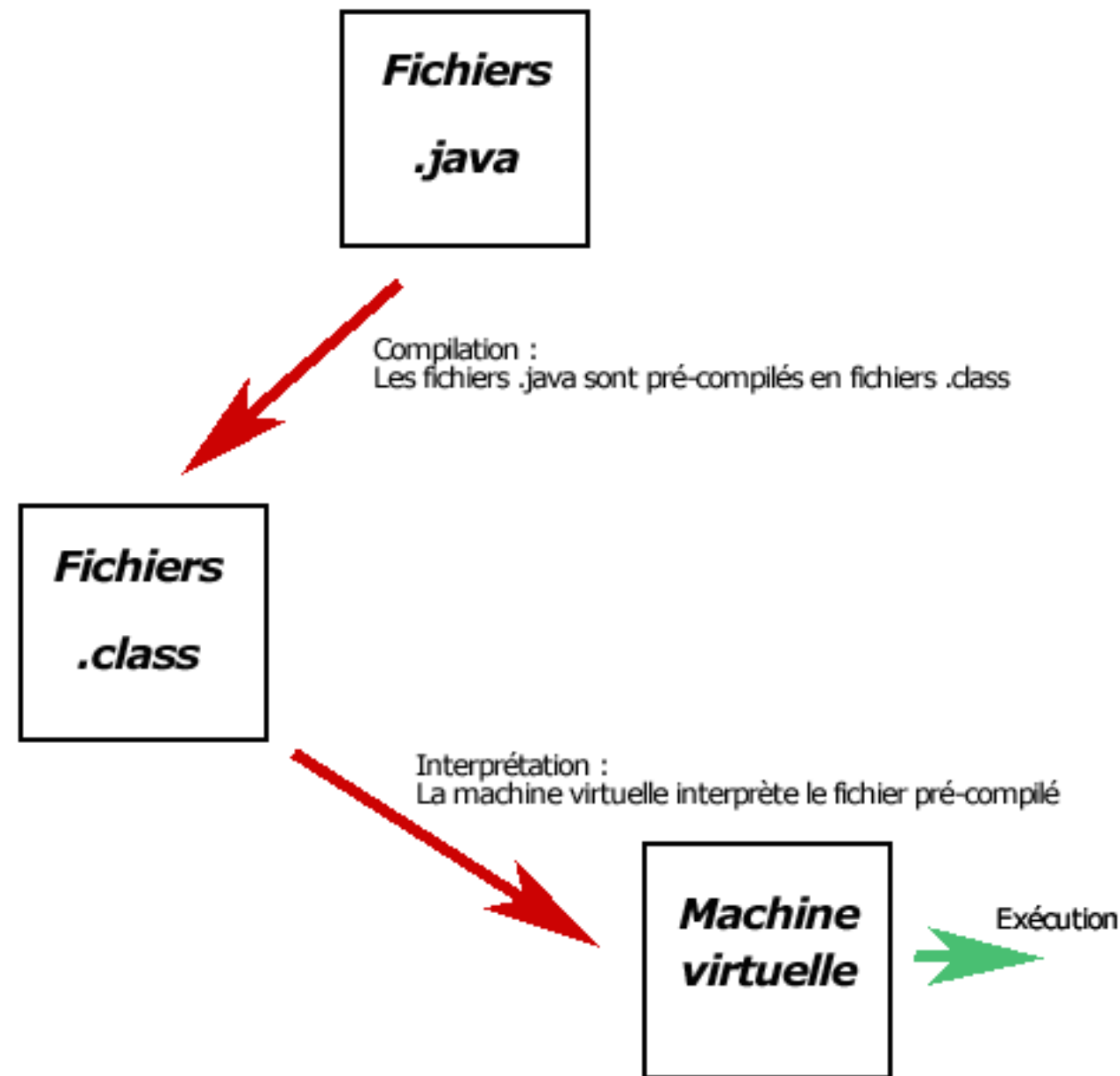
Les caractéristiques

Caractéristique	Signification
Langage interprété	Le code source est compilé en byte code et exécuté par la JVM
Portable	Le langage Java est indépendant de toute plateforme. Le code écrit peut s'exécuter n'importe où
Orienté objet	Java rassemble dans un même objet (classe) les propriétés et les fonctions qui s'y appliquent. Java est centré sur le paradigme orienté objet, utilisant des concepts tels que les classes , objets , héritage , polymorphisme , encapsulation et abstraction .
Fortement typé	Tous les attributs utilisés doivent avoir un type défini
Gestion de la mémoire	Java gère automatiquement l'allocation et la libération de la mémoire avec le Garbage Collector, ce qui réduit les risques de fuites de mémoire et d'erreurs liées à la gestion manuelle.
Multithreading	Permet l'utilisation de Threads qui sont des unités d'exécutions isolées. Le langage Java permet donc d'effectuer des traitements en parallèle

Les caractéristiques

Caractéristique	Signification
Sécurité	Java inclut des mécanismes pour éviter les failles courantes comme les dépassements de mémoire. Il dispose d'un modèle de sécurité robuste basé sur des classes et la gestion fine des droits d'accès.
Robustesse	Le système de gestion des erreurs de Java repose sur un mécanisme d'exceptions, ce qui permet d'écrire un code plus sûr et résistant aux bugs.
Extensibilité	Java est hautement extensible grâce à l'existence d'un écosystème riche de frameworks et de bibliothèques (Spring, Hibernate, Struts...).
Large communauté d'utilisateurs et Outils	Java bénéficie d'une large communauté de développeurs, d'un support étendu, et d'une gamme d'outils de développement tels qu'Eclipse, IntelliJ IDEA, et NetBeans. En raison de ces caractéristiques, Java est largement utilisé pour le développement d'applications d'entreprise, de systèmes embarqués, d'applications mobiles ...

Schéma d'exécution d'un code Java



Les principes de la programmation orientée objet

Encapsulation : C'est le fondement même de la programmation orientée objet. Il consiste à regrouper tout ce qui est lié à un objet au sein d'une classe. Cela implique donc d'intégrer chaque attribut et chaque méthode dans un seul emplacement.

Héritage : il consiste à intégrer les méthodes et les attributs d'une classe dans une autre classe. La classe de base est appelée classe mère et celle qui hérite est appelée la classe fille.

Polymorphisme : déclarer plusieurs fonctions avec le même nom, avec des attributs ou types de retour différents. Il permet de ne pas dupliquer de code, ce qui rend l'ensemble de la solution plus fluide et plus facile à maintenir.

Classe

Une classe est entité représentant un ensemble d'objets ayant la même structure et les mêmes comportements.

Syntaxe:

- public; private ou protected // portée de la classe
- abstract // classe impossible à instancier
- final // classe impossible à dériver
- class « nom de la classe »
- extends « nom de la classe mère »
- implements « nom d'interface » - { "corps de la classe" }

Exemple :

```
public class Article implements Serializable {  
    private static final long serialVersionUID = 7596696243276192383L;  
    private Long id;  
    private String code;  
    private String designation;  
    private String reference;  
    private Double prix;  
}
```

Package

Un **package** en Java est un « conteneur » utilisé pour regrouper les classes associées. Il représente un dossier dans un répertoire de fichiers. Les packages sont utilisés pour éviter les conflits de noms et pour écrire un code mieux maintenable.

Pour utiliser une classe ou un package de la bibliothèque, vous devez utiliser le mot-clé **import**

Exemple :

```
package net.ifri.model;
import java.io.Serializable;
import java.util.Date;

public class Article implements Serializable {
    /**
     * CODE
     */
}
```

Quelques bonnes pratiques de développement en JAVA

Le style de code

Bien indenter le code permet d'avoir un code clair et lisible. Cela permet aussi d'éviter les conflits avec les outils de versionning tels que SVN ou Git

```
/**
 * Mauvaise pratique
 */
public static File getPerformanceFile(String repName, String fileName) {
fileName = fileName + ".csv"; File fichier = new File(repName, fileName);
return fichier;}
```

```
/**
 * Bonne pratique
 */
public static File getPerformanceFile(String repName, String fileName) {
    fileName = fileName + ".csv";
    File fichier = new File(repName, fileName);
    return fichier;
}
```

Le style de code

Il est recommandé de mettre des accolades autour des structures de contrôle : if, for, while, try...

```
/**
 * Mauvaise pratique
 */
if (StringUtils.isNotBlank(bean.getQteStockMoyMensNat()))
    product.setQteStockMoyMensNat(Float.valueOf(bean.getQteStockMoyMensNat()));
else
    product.setQteStockMoyMensNat(null);
```

```
/**
 * Bonne pratique
 */
if (StringUtils.isNotBlank(bean.getQteStockMoyMensNat())) {
    product.setQteStockMoyMensNat(Float.valueOf(bean.getQteStockMoyMensNat()));
} else {
    product.setQteStockMoyMensNat(null);
}
```

Le style de code

Il est également préférable d'éviter les opérateurs ternaires qui sont difficiles à lire par certains développeurs. Cela rend le débogage du code fastidieux.

```
/**
 * Mauvaise pratique
 */
final Integer result = (isEnabled ()) ? getValidCode ( ) : (isBusy()) ? getBusyCode() : getErrorCode ( ) ;
```

```
/**
 * Bonne pratique
 */
Integer result = getErrorCode ();
if (isEnabled ()) {
    result = getValidCode ();
}

if (isBusy()) {
    result = getBusyCode();
}
```

Les métriques

Il s'agit d'indicateurs qui permettent de mesurer la qualité du logiciel.

Un bon code est clair, concis et facilement maintenable par d'autres développeurs.

Quelques indicateurs :

- **Le nombre de lignes de codes dans une classe**: La limite communément admise par classe est de 1000 lignes.
- **La longueur des méthodes**: La limite communément admise pour une méthode est de 100 lignes. Au delà, il faut un découpage en plusieurs méthodes et/ou classes de longueur raisonnable.
- **Le nombre de paramètres dans la signature d'une méthode**: La limite est de 7 par méthode. Plus une méthode comporte de paramètres, plus il est nécessaire de les regrouper dans un nouvel objet.

Les métriques

```
/**
 * Mauvaise pratique */
public void doSomething() { System.out.println("Hello world!");
// 200 copies omitted for brevity .}
```

```
/*
 * Bonne pratique ( java ) */
public void doSomething() {
    System.out.println("Hello world!");
    // 100 copies omitted for brevity
    doSubRoutine ( ) ;
}
public void doSubRoutine () {
    // 100 copies omitted for brevity
}
```


La complexité cyclomatique

Encore appelée mesure de McCabe, la complexité cyclomatique est une mesure représentant sous forme de graphe, le nombre de chemins qu'il est possible de suivre dans un algorithme.

La complexité cyclomatique s'incrémente à chaque instruction conditionnelle (if, else, case, for...) ajoutée à votre algorithme.

```
switch(x) {  
  case 1: return "a";  
  case 2: return "b";  
  case 3: return "c";  
  case 4: return "d";  
  case 5: return "e";  
  case 6: return "f";  
}  
return "other"; //complexité de 7
```

La complexité cyclomatique

Avantage: Un code simple à faible complexité cyclomatique est théoriquement plus facile à lire, à tester et à entretenir.

Inconvénient: Il ne faut pas rechercher systématiquement une complexité cyclomatique faible au risque de rendre le code moins lisible.

```
switch(x) {  
  case 1: return "a";  
  case 2: return "b";  
  case 3: return "c";  
  case 4: return "d";  
  case 5: return "e";  
  case 6: return "f";  
}  
return "other";//complexité de 7
```

```
var retValues = ["a", "b", "c", "d", "e", "f"];  
  
if (x > 0 && x <= 6) {  
  return retValues[x - 1];  
}  
return "other";//complexité de 3  
  
Code pas compréhensible cependant
```

Les métriques

Des outils OpenSource permettent de calculer les métriques de votre projet.
Exemple: SonarQube, Metrics, Checkstyle, EcEmma.

Focus sur le plugin eclipse « **EcEmma** » et présentation de SonarQube

Mise en place en TP

Le Pair Programming



Les design pattern en JAVA

Les design patterns

Les design patterns sont apparus dans le but de faciliter la maintenance et la réutilisabilité du code.

On est parti du constat que le nombre d'applications développées avec des technologies orientées objet augmente. Alors l'idée de réutiliser des techniques pour solutionner des problèmes courants a abouti au recensement d'un certain nombre de modèles connus sous le nom de motifs de conception (design patterns).

Ces modèles sont définis pour être utilisés avec un maximum de langages de développement orientés objet.

Origine des design patterns

L'origine des Design Patterns remonte au début des années 1970 avec les travaux de l'architecte Christopher Alexander qui remarqua que la phase de conception en architecture fait apparaître des problèmes récurrents.

Il chercha alors à résoudre l'ensemble de ces problèmes liés à des contraintes interdépendantes (solidité de la structure, étanchéité...).

Pour cela il établit un langage de 253 patterns, qui couvrent tous les aspects de la construction (comme par exemple la façon de concevoir une charpente).

Les Design Patterns trouvent leur origine donc dans le domaine du bâtiment et de la construction.

Origine des design patterns

A la suite dans les années 90, l'idée de Christopher Alexander va être reprise et étendue au domaine de la conception des logiciels.

Le concept de Design Pattern est développé dans un ouvrage publié en 1995 par une équipe composée de Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides. L'ouvrage s'intitule '**Design Patterns : Elements of Reusable Object-Oriented Software**'.

Cet ouvrage présente 23 Design Patterns qui font aujourd'hui référence dans le monde de l'informatique.

Les design patterns en Java

En Java, les design patterns sont des solutions éprouvées à des problèmes récurrents dans le développement logiciel.

Ils aident à concevoir des applications robustes, maintenables et évolutives.

Les design patterns sont généralement classés en trois catégories principales : **créationnels**, **structurels** et **comportementaux**.

Les catégories de design pattern

Il existe 3 grandes catégories ces modèles de conception :

- **les modèles créationnels (creational patterns)** : ils permettent d'instancier et de configurer des classes et des objets. Ils concernent la manière de créer des objets tout en masquant la logique complexe de création.
- **les modèles structurels (structural patterns)** : ils permettent d'organiser les classes d'une application. Ces modèles traitent de la composition des classes et des objets pour former des structures plus grandes.
- **les modèles comportementaux (behavioral patterns)** : ils permettent d'organiser les objets pour qu'ils collaborent entre eux. Ils s'intéressent à l'interaction entre les objets et à la manière de répartir les responsabilités.

Les modèles de création

Dans cette catégorie on distingue généralement 5 modèles :

Nom	Rôle
Singleton	Design pattern garantissant qu'une classe qui ne pourra avoir qu'une seule instance pendant l'exécution du programme et fournit un point d'accès global à celle-ci.
Factory	Design pattern définit une méthode pour créer des objets sans spécifier leur classe concrète.
Abstract Factory	Fournir une interface unique pour instancier des objets d'une même famille sans avoir à connaître les classes à instancier
Builder	Modèle utilisé pour la création d'objets complexes dont les différentes parties doivent être créées suivant un certain ordre ou algorithme spécifique.
Prototype	Création d'objet à partir d'un prototype

Le Singleton

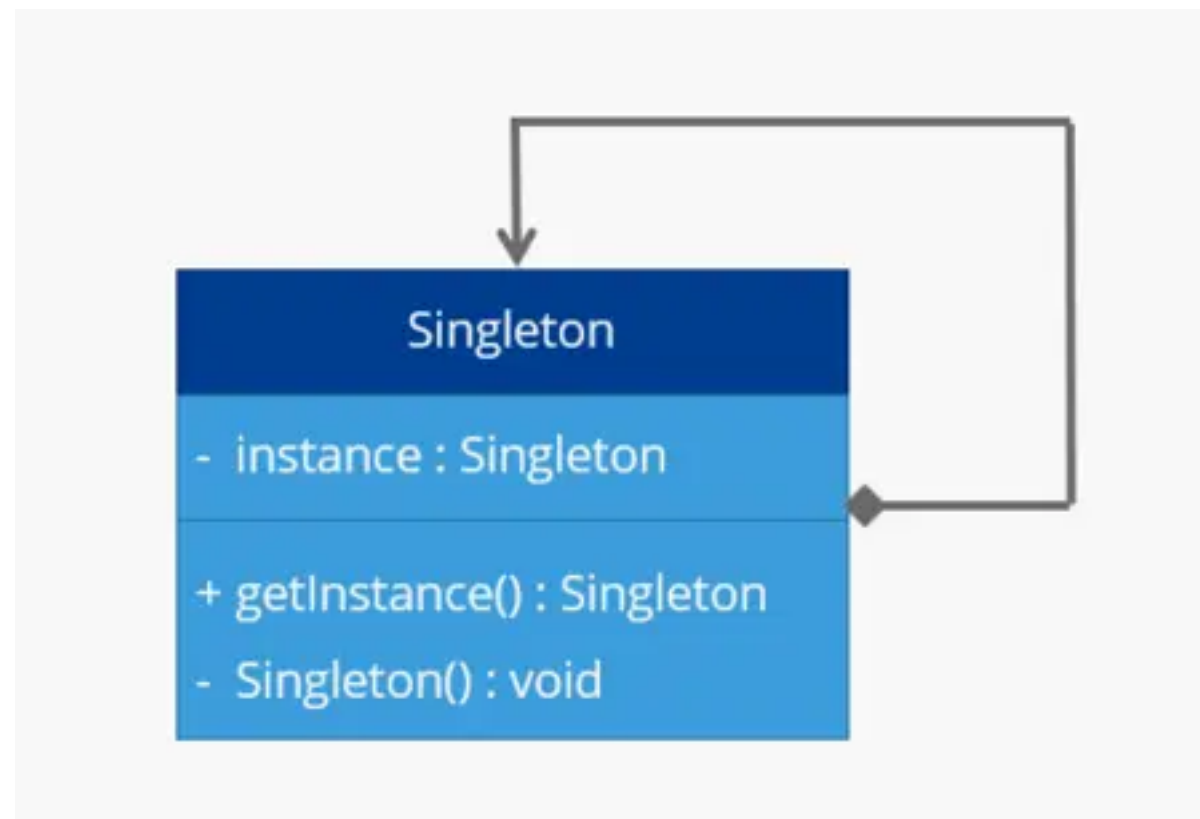
Un singleton est une classe qui ne peut avoir qu'une seule et unique instance dans un programme.

Un singleton est souvent pratique lorsqu'on souhaite manipuler des classes sans état qui effectuent toujours les mêmes traitements.

Ainsi une classe qui ne comporte que des méthodes statiques doit être transformée en un singleton.

Pour éviter toute instanciation extérieure à cette classe, il est nécessaire de rajouter un constructeur privé dans la classe Singleton.

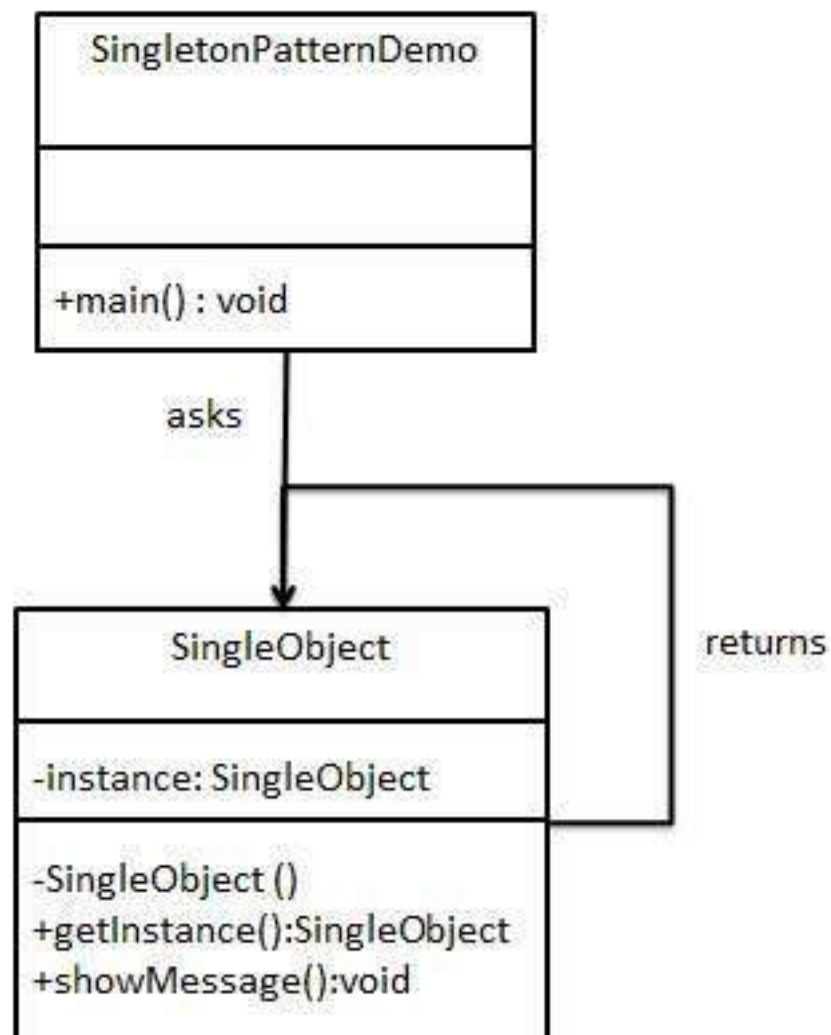
Représentation UML du design pattern Singleton



Le Singleton

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Le Singleton



Le pattern Builder

Le pattern Builder permet de construire des objets complexes à partir d'objets simples en se basant sur une approche pas-à-pas.

Le pattern Builder est utilisé pour la création d'objets complexes dont les différentes parties doivent être créées suivant un certain ordre ou algorithme spécifique. Une classe externe contrôle l'algorithme de construction.

Il permet ainsi de dissocier la construction d'un objet de sa représentation, afin que le même processus de construction ait la possibilité de créer des représentations différentes.

Le pattern Builder

Ce design pattern sera préconisé dans les situations où au moins :

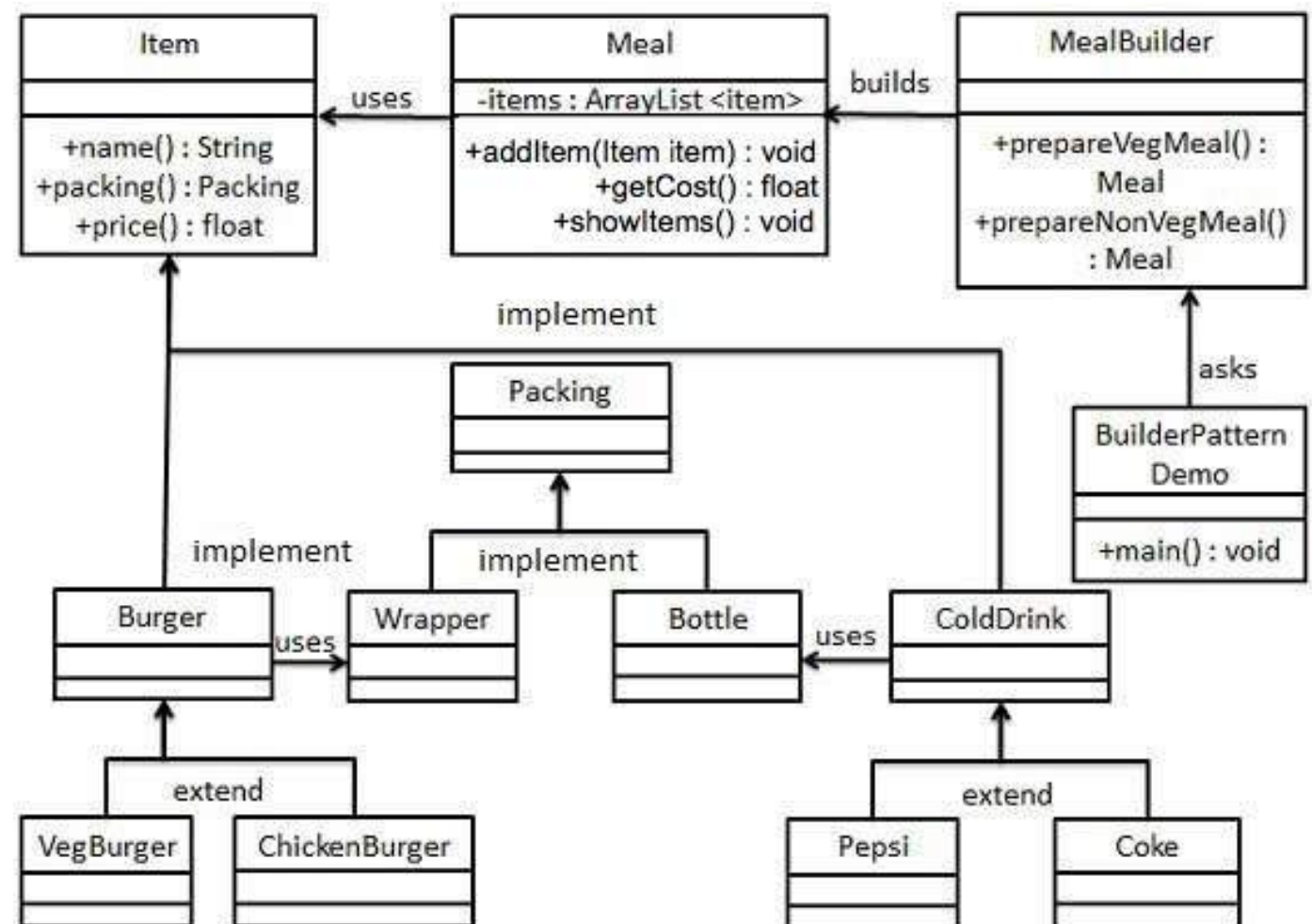
- l'objet final est imposant, et sa création complexe ;
- beaucoup d'arguments doivent être passés à la construction de l'objet, afin d'avoir un design lisible ;
- certains de ces arguments sont optionnels (par ex. : un bateau n'a pas forcément de capitaine, mais toujours une taille), ou ont plusieurs variations (la taille peut par exemple être passée en mètres ou en pieds).

Le pattern Builder

TP d'implémentation : Nous avons considéré le cas d'un restaurant de restauration rapide où un repas typique pourrait être un hamburger et une boisson fraîche. Le burger peut être un burger végétarien ou un burger au poulet et sera emballé dans un emballage. La boisson froide peut être un coca ou un pepsi et sera emballée dans une bouteille.

Nous allons créer une interface `Item` représentant des produits alimentaires tels que des hamburgers et des boissons froides et des classes concrètes implémentant l'interface `Item` et une interface `Packing` représentant l'emballage de produits alimentaires et des classes concrètes implémentant l'interface `Packing` car le hamburger serait emballé dans un emballage et une boisson froide, serait emballé en bouteille.

Nous créons ensuite une classe `Meal` ayant `ArrayList of Item` et un `MealBuilder` pour créer différents types d'objets `Meal` en combinant `Item`. `BuilderPatternDemo`, notre classe de démonstration utilisera `MealBuilder` pour créer un repas.



Le pattern Builder



Le pattern Builder

Etape 1: Création des interfaces Packing et Item

```
public interface Packing {  
    public String pack();  
}
```

```
public interface Item {  
    public String name();  
    public Packing packing();  
    public float price();  
}
```

Etape 2: Création des classes Wrapper et Bottle implémentant l'interface Packing

```
public class Wrapper implements Packing {  
    @Override  
    public String pack() {  
        return "Wrapper";  
    }  
}
```

```
public class Bottle implements Packing {  
    @Override  
    public String pack() {  
        return "Bottle";  
    }  
}
```

Le pattern Builder

Etape 3: Création des classes Burger et ColdDrink implémentant l'interface Item

```
public abstract class Burger implements Item {  
  
    @Override  
    public Packing packing() {  
        return new Wrapper();  
    }  
  
    @Override  
    public abstract float price();  
}
```

```
public abstract class ColdDrink implements Item {  
  
    @Override  
    public Packing packing() {  
        return new Bottle();  
    }  
  
    @Override  
    public abstract float price();  
}
```

Le pattern Builder

Etape 4: Création des classes VegBurger, ChickenBurger, Pepsi et Coca qui étendent Burger et ColdDrink

```
public class VegBurger extends Burger {  
  
    @Override  
    public float price() {  
        return 25.0f;  
    }  
  
    @Override  
    public String name() {  
        return "Veg Burger";  
    }  
  
}
```

```
public class ChickenBurger extends Burger {  
  
    @Override  
    public float price() {  
        return 50.5f;  
    }  
  
    @Override  
    public String name() {  
        return "Chicken Burger";  
    }  
  
}
```

Le pattern Builder

Etape 4: Création des classes VegBurger, ChickenBurger, Pepsi et Coca qui étendent Burger et ColdDrink

```
public class Coke extends ColdDrink {  
  
    @Override  
    public float price() {  
        return 30.0f;  
    }  
  
    @Override  
    public String name() {  
        return "Coke";  
    }  
  
}
```

```
public class Pepsi extends ColdDrink {  
  
    @Override  
    public float price() {  
        return 35.0f;  
    }  
  
    @Override  
    public String name() {  
        return "Pepsi";  
    }  
  
}
```


Le pattern Builder

Etape 6: Création de la classe Meal

```
import java.util.ArrayList;
import java.util.List;

public class Meal {
    private List<Item> items = new ArrayList<Item>();

    public void addItem(Item item) {
        items.add(item);
    }

    public float getCost() {
        float cost = 0.0f;
        for (Item item : items) {
            cost += item.price();
        }
        return cost;
    }

    public void showItems() {
        for (Item item : items) {
            System.out.println("Item : " + item.name() + ", Packing : " + item.packing().pack() + ", Price : " + item.price());
        }
    }
}
```


Le pattern Builder

Etape 6: Création du builder MealBuilder qui est le pattern chargé de créer les objets Meal

```
public class MealBuilder {  
    public Meal prepareVegMeal () {  
        Meal meal = new Meal();  
        meal.addItem(new VegBurger());  
        meal.addItem(new Coke());  
        return meal;  
    }  
    public Meal prepareNonVegMeal () {  
        Meal meal = new Meal();  
        meal.addItem(new ChickenBurger());  
        meal.addItem(new Pepsi());  
        return meal;  
    }  
}
```

Etape 7: La classe BuildPatternDemo permettant de lancer le programme

```
public class BuilderPatternDemo {  
    public static void main(String[] args) {  
        MealBuilder mealBuilder = new MealBuilder();  
        Meal vegMeal = mealBuilder.prepareVegMeal();  
        System.out.println("Vegetarian Meal");  
        vegMeal.showItems();  
        System.out.println("Total Cost: " + vegMeal.getCost());  
  
        Meal nonVegMeal = mealBuilder.prepareNonVegMeal();  
        System.out.println("Chicken Meal");  
        nonVegMeal.showItems();  
        System.out.println("Total Cost: " + nonVegMeal.getCost());  
    }  
}
```